

Querying Multiple Tables (Joins & Aggregations)

Master the art of combining data from multiple tables and extracting meaningful insights.



Querying Multiple Tables (Joins & Aggregations)

Joins, Aggregations & Complex Queries

Unlock meaningful insights by combining data and calculating summaries with advanced SQL techniques. This presentation will guide you through essential concepts for powerful database interactions.

01

SQL Joins

Inner vs. Outer Joins

02

Aggregate Functions

Count, Sum, Avg, Min, Max

03

Grouping Data

Group By, Having Clauses

04

Subqueries & Aliases

Enhancing query readability and functionality

Understanding Joins: Connecting Your Data

SQL Joins are fundamental for combining data from two or more tables based on related columns, typically primary and foreign keys. This allows you to retrieve a comprehensive dataset from disparate sources.

1

INNER JOIN

Returns only rows with matching values in **both** tables. Think of it as the intersection of two datasets, showing only common records.

2

LEFT (OUTER) JOIN

Returns **all** records from the left table, plus matched records from the right. If no match is found on the right, NULL values are returned for right table columns.

3

RIGHT (OUTER) JOIN

Returns **all** records from the right table, plus matched records from the left. NULLs appear for unmatched left table columns.

4

FULL (OUTER) JOIN

Returns all records where there's a match in **either** the left or right table. It's the union of both tables, with NULLs for unmatched sides.

SQL Joins: Practical Examples

Let's dive into practical SQL examples to see how different join types combine data from two tables, `Table_A` and `Table_B`.

Setup: Tables and Data

```
-- Create Tables
CREATE TABLE Table_A (ID INT, Name VARCHAR(10));
CREATE TABLE Table_B (ID INT, Name VARCHAR(10));

-- Insert Data
INSERT INTO Table_A VALUES (1, 'Alpha'), (2, 'Beta'); -- 2 is only in A
INSERT INTO Table_B VALUES (1, 'Alpha'), (3, 'Gamma'); -- 3 is only in B
```

INNER JOIN

Returns only rows with matching IDs in both tables.

```
SELECT
  A.ID AS ID_A,
  A.Name AS Name_A,
  B.ID AS ID_B,
  B.Name AS Name_B
FROM Table_A A
INNER JOIN Table_B B ON A.ID = B.ID;
```

LEFT (OUTER) JOIN

Returns all records from the Left table (`Table_A`), and matched records from the Right table (`Table_B`). If no match, `Table_B` columns are `NULL`.

```
SELECT
  A.ID AS ID_A,
  A.Name AS Name_A,
  B.ID AS ID_B,
  B.Name AS Name_B
FROM Table_A A
LEFT JOIN Table_B B ON A.ID = B.ID;
```

RIGHT (OUTER) JOIN

Returns all records from the Right table (`Table_B`), and matched records from the Left table (`Table_A`). If no match, `Table_A` columns are `NULL`.

```
SELECT
  A.ID AS ID_A,
  A.Name AS Name_A,
  B.ID AS ID_B,
  B.Name AS Name_B
FROM Table_A A
RIGHT JOIN Table_B B ON A.ID = B.ID;
```

FULL (OUTER) JOIN

Returns all records when there is a match in either the left or right table, including non-matching rows from both sides. Unmatched columns will be `NULL`.

```
SELECT
  COALESCE(A.ID, B.ID) AS Final_ID,
  A.Name AS Name_A,
  B.Name AS Name_B
FROM Table_A A
FULL JOIN Table_B B ON A.ID = B.ID;
```

Aliases for Cleaner and More Efficient Queries

Using an **Alias (AS)** allows you to assign a temporary, shorthand name to tables or columns. This greatly improves the readability of your SQL code, especially in complex queries involving multiple tables.

Benefit:

- Reduces the amount of typing required.
- Makes complex queries much easier to understand at a glance.
- Prevents ambiguity when columns have the same name in different tables.

Leveraging Aggregate Functions for Data Summaries

Aggregate functions perform calculations on a set of rows and return a single summary value, providing valuable insights into your data.



COUNT()

Returns the number of rows in a specified group or for the entire table. Useful for counting records, such as the total number of students.



SUM()

Calculates the sum of all values in a numeric column. Perfect for totaling quantities like sales figures or credits.



AVG()

Computes the average of a numeric column's values. Helps in understanding central tendencies, such as average course credits.



MIN() / MAX()

Retrieves the smallest (MIN) or largest (MAX) value from a specified column. Useful for identifying extremes, like the minimum or maximum credits offered.

Example Queries:

```
SELECT COUNT(*) FROM Courses;
```

```
SELECT SUM(Credits) FROM Courses;
```

```
SELECT AVG(Credits) FROM Courses;
```

```
SELECT MIN(Credits) FROM Courses;
```

```
SELECT MAX(Credits) FROM Courses;
```

Grouping Data with GROUP BY

The **GROUP BY** clause arranges rows that have identical values into summary rows, often used with aggregate functions to perform calculations on each group.

Key Rule:

Any column included in your **SELECT** statement that is **not** an aggregate function must also be present in your **GROUP BY** clause. This ensures that the results are appropriately grouped and aggregated.

Scenario: "Count how many students are enrolled in each specific course."

Example:

```
SELECT CourseID, COUNT(StudentID) AS StudentCount
FROM Enrollments
GROUP BY CourseID;
```

Filtering Grouped Data with HAVING

HAVING vs. WHERE: A Critical Distinction

- **WHERE:** Filters individual rows **before** they are grouped. It cannot be used with aggregate functions directly.
- **HAVING:** Filters groups of rows **after** the grouping and aggregation have occurred. It is specifically designed to work with aggregate functions.

Scenario: "Show me only those courses that have more than 5 students enrolled."

Here, we first group by `CourseID` and count students, then filter those groups based on the count.

Example:

```
SELECT CourseID, COUNT(StudentID) AS StudentCount
FROM Enrollments
GROUP BY CourseID
HAVING COUNT(StudentID) > 15;
```

When to use HAVING:

Use **HAVING** when your filtering condition depends on the result of an aggregate function (e.g., `COUNT()`, `SUM()`, `AVG()`) applied to a group.

Subqueries: Nesting Queries for Dynamic Filtering

Definition:

A **subquery** (or inner query) is a query embedded within another SQL query (the outer query). It executes first, and its result is then used by the outer query.

Primary Use Case:

Subqueries are invaluable when you need a specific value or set of values to filter your main query, but that value isn't known beforehand and must be derived from another query.

Scenario: "Find all courses that offer the maximum number of credits available in the entire catalog."

The inner query first determines the `MAX(Credits)`, and the outer query then uses this value to find the corresponding courses.

Example:

```
SELECT *  
FROM Courses  
WHERE Credits = (SELECT MAX(Credits) FROM Courses);
```

Hands-on Activity: Apply Your SQL Knowledge

Let's put your new SQL skills to the test with these practical exercises. Use the concepts we've covered to solve the following tasks.

1 Task 1:

Retrieve a list of all enrolled students. Show the **Student Name**, the **Course Name** they are taking, and the **Department ID** for that course.

2 Task 2:

Calculate the total number of **Credits** each student is taking. Display the **Student Name** and their **Total Credits**.

3 Task 3:

Find the number of courses offered by **Department ID 3**. Display the DepartmentID and the count of courses.

4 Task 4:

List all **Courses** that currently have **zero students** enrolled in them.

5 Task 5:

List all courses and the number of students enrolled in them, ordered from **most popular** (highest student count) to **least popular**.

Answer Script: Solutions to Practice Tasks

Here are the SQL queries to solve the previous hands-on activities. Review these solutions to deepen your understanding.

1	Task 1: <pre>SELECT S.Name AS StudentName, C.Name AS CourseName, C.DepartmentID FROM Students S JOIN Enrollments E ON S.StudentID = E.StudentID JOIN Courses C ON E.CourseID = C.CourseID;</pre>
2	Task 2: <pre>SELECT S.Name, SUM(C.Credits) AS TotalCredits FROM Students S JOIN Enrollments E ON S.StudentID = E.StudentID JOIN Courses C ON E.CourseID = C.CourseID GROUP BY S.Name;</pre>
3	Task 3: <pre>SELECT DepartmentID, COUNT(CourseID) AS CourseCount FROM Courses WHERE DepartmentID = 3 GROUP BY DepartmentID;</pre>
4	Task 4: <pre>SELECT C.Name AS CourseName FROM Courses C LEFT JOIN Enrollments E ON C.CourseID = E.CourseID WHERE E.EnrollmentID IS NULL;</pre>
5	Task 5: <pre>SELECT C.Name AS CourseName, COUNT(E.StudentID) AS StudentCount FROM Courses C JOIN Enrollments E ON C.CourseID = E.CourseID GROUP BY C.Name ORDER BY StudentCount DESC;</pre>